MBLP131 · Hafta 14 · Ders Notları

İşletim Sistemleri Temelleri

Kabuk Betiklerine Giriş

Bir kez yazın, kaydedin, bin kez çalıştırın.

Öğretim üyesi: Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

Bu haftanın sonunda yapabilmeniz gerekenler

1. Kabuk betiğinin ne olduğunu, neden yararlı olduğunu ve bir betik çalıştırıldığında ne olduğunu (fork, exec, shebang) açıklamak.
2. Shebang satırı olan bir betik yazmak, onu `chmod +x` ile çalıştırılabilir yapmak ve dört olası yolla çalıştırmak.
3. Değişkenleri, üç tür tırnaklamayı, komut yerine koymayı `$( )` ve tamsayı aritmetiğini `$(( ))` kullanmak.
4. Argümanları (`$0 $1 $# $@`) ve klavye girdisini (`read`) okumak.
5. Çıkış durumlarını, sayılar, dizgiler ve dosyalar için `test` / `[ ]` yapısını ve `if / elif / else` deyimini kullanmak.
6. `for` ve `while` döngüleri ile basit fonksiyonlar yazmak.
7. Bir betikte `bash -n` ve `bash -x` ile hata ayıklamak ve en sık görülen hata iletilerini tanımak.
8. İki eksiksiz otomasyon betiğini okuyup uyarlamak: `backup.sh` ve `report.sh`.

İçindekiler

1 13. Haftadan 14. Haftaya 1

2 Neden Betik Yazarız? 2

3 İlk Betiğiniz 2

4 Değişkenler, Tırnaklar ve Girdi 4

5 Çıkış Durumu ve Testler 8

6 Karar Verme: `if` 9

7 Döngüler 12

8 Sözdizimi Başvuru Tablosu 14

9 İki Gerçek Otomasyon Betiği 15

10 Sık Görülen Hatalar ve Anlamları 17

11 Özet 19

12 Sözlükçe 20

13 Kendiniz Deneyin 20

14 Tekrar Soruları 20

15 Sırada Ne Var: 15. Hafta 21

16 Lab Föyü: Notlandırılan Betik Yazma Alıştırmaları 22

17 Lab Föyü Çözümleri 23

1 13. Haftadan 14. Haftaya

Geçen hafta **süreçlere** (processes) komut satırından baktık: her komut bir PID'si ve bir ebeveyni olan bir süreç olarak çalışır, `ps` ve `top` bunları gösterir, işler ön plan ile arka plan arasında taşınabilir ve `kill` sinyal gönderir. 9–12. haftalarda komutların kendisini öğrendik: dizinlerde gezinme, dosya işlemleri, izinler, yönlendirme ve boru hatları.

Şimdiye kadar her komutu elle yazdık. Bu hafta komutları bir dosyaya kaydediyor ve onları kabuğun bizim yerimize çalıştırmasını sağlıyoruz. Böyle bir dosyaya **kabuk betiği** (shell script) denir. Betik de yalnızca bir programdır: onu çalıştırdığınızda kabuk, tıpkı `ls` ya da `sleep` için yaptığı gibi onun için yeni bir süreç oluşturur ve 13. haftada gördüğümüz her şey ona da uygulanır.

Bu notları nasıl izlemelisiniz? Tüm örnek betikler bu notların yanındaki `code/` klasöründedir. Temiz bir laboratuvar klasörü oluşturun, betikleri kopyalayın ve `~/lab/scripts` içinde çalışın:

```
$ bash mcmp117_lab_setup.sh          # ~/lab'ı yeniden oluşturur (herkes için aynı
    dosyalar)
$ cp *.sh ~/lab/scripts/             # bunu hafta14/code klasöründe çalıştırın
$ cd ~/lab/scripts
```

Bu notlardaki her çıktı, deneme makinemiz `okan-lab` üzerinde sıradan kullanıcı `student` olarak, `LC_ALL =C.UTF-8` ayarıyla yakalanmıştır. Sizin makinenizde tarihler, saatler, dosya boyutları, PID'ler ve kullanıcı adları farklı olacak; `ls` ya da `sort` çıktısının sırası da yerel ayara (locale) göre değişebilir. Türkçe yerel ayarlı bir sistemde (`LANG =tr_TR.UTF-8`) birçok ileti Türkçe görünür ve sıralama farklı olabilir;

ders çıktıları `LC_ALL =C.UTF-8` ile alınmıştır. Kabuk GNU `bash` 5.2'dir. Örnek betiklerdeki yorumlar ve program iletileri, İngilizce şubeyle ortak olduğu ve Linux belgelerinin dili İngilizce olduğu için İngilizcedir (`variables.sh` bu yüzden ders kodu olarak `MCMP117` yazdırır); kendi kopyalarınızda Türkçe yorum yazabilirsiniz.

2 Neden Betik Yazarız?

Her sabah bir klasörü yedeklediğinizi, bir günlük (log) dosyasında hata aradığınızı ve dünkü web isteklerini saydığınızı düşünün: doğru sırayla, doğru yollarla yazılması gereken altı yedi komut. Bunları elle yazmak yavaştır, yanlış yapmaya açıktır, bir iş arkadaşınıza anlatması zordur ve siz uyurken olanaksızdır. Bir betik bu dört sorunun hepsini çözer:

- **Tekrarlanabilirlik:** tek bir komut, `./backup.sh`, her seferinde aynı adımları yapar.
- **Daha az hata:** bir yazım hatası dosyada bir kez düzeltilir ve düzeltilmiş kalır.
- **Belgeleme:** yorumlarıyla birlikte betiğin kendisi yordamın tanımıdır.
- **Otomasyon:** `cron` gibi bir zamanlayıcı, klavye başında kimse yokken bir betiği belirli saatlerde başlatabilir.

Yararlı bir pratik kural: aynı komut dizisini üçüncü kez yazıyorsanız bir betik yazın. Sistem yöneticileri, DevOps mühendisleri ve gömülü Linux geliştiricileri her gün küçük betikler yazar; Linux'un kendisi de açılırken birçok kabuk betiği çalıştırır.

3 İlk Betiğiniz

3.1 Bir betiğin anatomisi

Betik düz bir metin dosyasıdır. Onu herhangi bir metin düzenleyiciyle oluşturun, örneğin `nano hello.sh` (Ctrl+O ve Enter ile kaydedin, Ctrl+X ile çıkın):

```
#!/bin/bash
# hello.sh -- my first shell script
# Run: bash hello.sh   or: chmod +x hello.sh; ./hello.sh
echo "Hello from a shell script!"
echo "I am running as: $(whoami)"
echo "Current folder:  $(pwd)"
```

- **1. satır, shebang:** `#!` ve ardından dosyayı çalıştırması gereken programın mutlak yolu. `#!/bin/bash` bash'i seçer; `#!/usr/bin/python3` bir Python programını tam olarak aynı yolla çalıştırırdı. Bu satır *yalnızca* en ilk satırda özeldir.
- **Yorumlar:** # işaretinden sonraki her şey bash tarafından yok sayılır ve insanlar için yazılmıştır.
- **Komutlar:** tam olarak istemde (prompt) yazacağınız şeyler. `$(whoami)` bir *komut yerine koymadır* (command substitution, Bölüm 4.2).

`.sh` uzantısı yalnızca insanlar için bir alışkanlıktır. Linux uzantılara bakmaz (7. hafta: dosya türleri); çalıştırma iznine ve ilk satıra bakarak karar verir.

3.2 Çalıştırmak

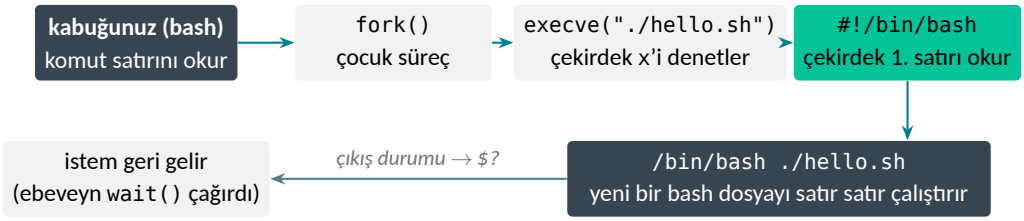
```
student@okan-lab:~/lab/scripts$ bash hello.sh
Hello from a shell script!
I am running as: student
Current folder:  /home/student/lab/scripts
student@okan-lab:~/lab/scripts$ ./hello.sh
bash: ./hello.sh: Permission denied
student@okan-lab:~/lab/scripts$ echo $?
126
student@okan-lab:~/lab/scripts$ ls -l hello.sh
-rw-rw-r-- 1 student student 204 Oct 12 17:18 hello.sh
student@okan-lab:~/lab/scripts$ chmod +x hello.sh
student@okan-lab:~/lab/scripts$ ls -l hello.sh
-rwxrwxr-x 1 student student 204 Oct 12 17:18 hello.sh
student@okan-lab:~/lab/scripts$ ./hello.sh
Hello from a shell script!
I am running as: student
Current folder:  /home/student/lab/scripts
student@okan-lab:~/lab/scripts$ hello.sh
bash: hello.sh: command not found
```

Ne oldu?

- `bash hello.sh` bash'i başlatır ve dosya adını ona argüman olarak verir. Bash dosyayı yalnızca *okur*; bu yüzden okuma izni yeterlidir.
- `./hello.sh` çekirdekten dosyayı *çalıştırmasını* ister; bunun için `x` izni gerekir. Dosya `-rw-rw-r--` kipiyle oluşturulmuştu (11. haftanın varsayılan umask değeri 002 şunu verir: $666 - 002 = 664$); sonuç `Permission denied` ve çıkış durumu 126'dır ("bulundu ama çalıştırılamıyor").
- `chmod +x hello.sh` komutu kullanıcı, grup ve diğerleri için `x` iznini ekler: $664 \rightarrow 775$, yani `rwxrwxr-x`. Artık `./hello.sh` çalışır.
- `./` olmadan `hello.sh` *bulunamaz*: kabuk komutları yalnızca PATH değişkeninde listelenen dizinlerde arar (`echo $PATH` ile deneyin) ve geçerli dizin bilerek bunlardan biri değildir. `./` "*bu* dizindeki dosya" demektir.

3.3 Perde arkasında neler olur?

`./hello.sh` çalıştırmak 3., 11. ve 13. haftaların mekanizmalarını kullanır:



Şekil 1: Bir betiği çalıştırmak: fork, exec, shebang, yeni bir yorumlayıcı süreci ve ebeveyne dönen çıkış durumu.

Kabuk bir çocuk süreç çatallar (`fork()`, 3. hafta) ve çocuk dosya üzerinde `execve()` çağırır. Çekirdek çalıştırma iznini denetler (11. hafta). Metin dosyası makine kodu değildir; bu yüzden çekirdek dosyanın ilk iki baytına bakar: `#!` şu anlama gelir: "bu satırda adı verilen programı başlat ve ona betiğin yolunu ver". Böylece

çekirdek aslında `/bin/bash ./hello.sh` komutunu çalıştırır. Ebeveyn kabuk bekler; betik bittiğinde çıkış durumu (exit status) ebeveyne döner ve `$?` içinde kullanılabilir.

Betik bir çocuk süreçte çalışır

Bir betiğin *içinde* ayarladığınız değişkenler ve çalıştırdığınız `cd` komutları yalnızca çocuk süreci değiştirir. Betik bittiğinde terminaliniz tam olarak bıraktığınız yerdedir. (Bir dosyayı *geçerli* kabukta çalıştırmak için `source dosya` ya da `. dosya` kullanılır; `~/.bashrc` bu şekilde okunur.)

3.4 Bir betiği çalıştırmanın dört yolu

Komut	Gerekenler	Nerede çalışır	Tipik kullanım
`bash hello.sh`	`r`	yeni bir bash süreci	hızlı deneme, x biti yok
`./hello.sh`	r+x, shebang	yeni bir süreç	bu dersteki olağan yol
`hello.sh`	r+x, dizin PATH'te	yeni bir süreç	kurulu araçlar (ör. `~/bin`)
`source hello.sh`	`r`	*geçerli* kabuk	değişken ayarlayan dosyalar

4 Değişkenler, Tırnaklar ve Girdi

4.1 Değişkenler

Bir **değişken** (variable) atamayla oluşturulur ve `$` ile kullanılır:

```
student@okan-lab:~/lab/scripts$ name="Ayse"
student@okan-lab:~/lab/scripts$ echo $name
Ayse
student@okan-lab:~/lab/scripts$ echo "Hello, $name!"
Hello, Ayse!
student@okan-lab:~/lab/scripts$ name = Ayse
bash: name: command not found
```

Kurallar kısa ama katıdır:

- **`=` işaretinin çevresinde boşluk yok.** Bash sözcükleri boşluklardan ayırır; bu yüzden `name = Ayse` , `=` ve `Ayse` argümanlarıyla çalıştırılan `name` komutudur; sonuç `name: command not found` olur. (Ubuntu'da *command-not-found* yardımcısı nedeniyle ileti daha uzundur; anlamı aynıdır.)
- Adlar harf, rakam ve `_` içerir, rakamla başlamaz ve büyük/küçük harfe duyarlıdır (`name` ile `NAME` farklıdır).
- `${name}` uzun biçimdir; adın ardından harf geliyorsa gereklidir: `"${name}s"`.
- Her değer metindir. `count=5` bir sayı değil, "5" metnini saklar; aritmetik için `$(( ))` gerekir.
- Tanımsız bir değişken yalnızca boştur; bash şikâyet etmez.
- Gelenek: ortam değişkenleri için BÜYÜK harfli adlar (`HOME`, `USER`, `PATH`, `SHELL`), kendi değişkenleriniz için küçük harfli adlar.

4.2 Komut yerine koyma ve aritmetik

```bash
#!/bin/bash
# variables.sh -- variables, command substitution and arithmetic
name="Ayse"                     # NO spaces around the = sign
course=MCMP117
week=14
echo "Student: $name, course: $course, week $week"

today=$(date +%Y-%m-%d)         # $( ) = the OUTPUT of a command
lines=$(wc -l < ~/lab/docs/syllabus.txt)
echo "Today is $today; the syllabus has $lines lines"

left=$((15 - week))             # $(( )) = integer arithmetic
echo "Weeks left until the final exam: $left"
echo "7 / 2 = $((7 / 2)) remainder $((7 % 2))   (integers only!)"
```

```
student@okan-lab:~/lab/scripts$ echo "Files in docs: $(ls ../docs | wc -l)"
Files in docs: 6
student@okan-lab:~/lab/scripts$ echo $((3 + 4 * 2))
11
student@okan-lab:~/lab/scripts$ ./variables.sh
Student: Ayse, course: MCMP117, week 14
Today is 2026-10-12; the syllabus has 15 lines
Weeks left until the final exam: 1
7 / 2 = 3 remainder 1   (integers only!)
```

- **Komut yerine koyma** (command substitution) `$(command)` komutu çalıştırır ve çıktısını (sondaki satır sonu olmadan) yerleştirir. Ters tırnaklı eski biçim aynı şeyi yapar ama okunması ve iç içe yazılması daha zordur. `wc -l < file` kullanımına dikkat edin: girdi yönlendirmesiyle (12. hafta) wc dosya adı olmadan yalnızca sayıyı yazar — `$( )` içinde tam istediğimiz şey.
- **Aritmetik genişletme** (arithmetic expansion) `$((expression))` tamsayılarla, `+ - * / %` ve parantezlerle, olağan işlem önceliğiyle hesap yapar: $3 + 4 \times 2 = 11$ (yukarıdaki döküme bakın). İçeride değişken adlarının önündeki $ isteğe bağlıdır.
- **Yalnızca tamsayılar:** bash 64 bitlik tamsayı aritmetiği kullandığı için $7/2$ sonucu 3, $7 \bmod 2 = 1$ olur. Ondalıklı hesap için awk (`report.sh`'de olduğu gibi) ya da hesap makinesi `bc` kullanın.

4.3 Tırnaklar

```bash
#!/bin/bash
# quotes.sh -- double quotes, single quotes, backslash, no quotes
name="Ayse"
file="final report.txt"         # a file name that contains a space
# shellcheck disable=SC2016     # (on purpose)
echo 'Single: Hello $name'      # '...' : everything is literal
echo "Double: Hello $name"      # "..." : $var and $( ) expanded
echo "Escaped: it costs \$5"    # \$ : one literal dollar sign
cd ~/lab/docs || exit 1
wc -l "$file"                   # quoted:   ONE argument
# shellcheck disable=SC2086     # (on purpose)
wc -l $file                     # unquoted: TWO arguments
```

```
student@okan-lab:~/lab/scripts$ ./quotes.sh
Single: Hello $name
Double: Hello Ayse
Escaped: it costs $5
2 final report.txt
wc: final: No such file or directory
wc: report.txt: No such file or directory
0 total
```

Biçim	`$var`, `$( )` genişletilir mi?	Boşluklar korunur mu?
`"çift tırnak"`	evet	evet — tek sözcük
`'tek tırnak'`	hayır, her şey olduğu gibi	evet — tek sözcük
ters eğik çizgi `\$`	*tek* karakteri olduğu gibi bırakır	—
tırnaksız	evet	hayır — sözcüklere bölünür, * genişletilir

`quotes.sh`'nin son iki satırı, gerçek betiklerdeki en yaygın hatayı gösterir. `file` değişkeni bir boşluk içerir. Tırnaklıyken `"$file"` tek bir argümandır ve `wc` 2 satır sayar. Tırnaksızken değer `final` ve `report.txt` adlı iki argümana bölünür ve `wc` iki olmayan dosya bildirir. **Kural: bir nedeniniz yoksa her `$değişkenin` ve her `$( )` ifadesinin çevresine çift tırnak koyun.**

İki `# shellcheck disable` satırı, tam da bu sorunlar için uyaran ücretsiz bir araç olan **ShellCheck** içindir (`https://www.shellcheck.net` ya da `sudo apt install shellcheck`). Bu satırlar araca hataların bilerek yapıldığını söyler. Bu haftanın diğer tüm örnek betikleri ShellCheck'ten uyarısız geçer.

4.4 Konumsal parametreler

Betik adından sonra yazılan sözcükler onun **argümanlarıdır** (arguments). Betiğin içinde bunlar `$1`, `$2`, … olarak görünür; bunlara **konumsal parametreler** (positional parameters) denir:

```
#!/bin/bash
# args.sh -- positional parameters: the words typed after the script name
echo "Script name  \$0 = $0"
echo "First arg    \$1 = $1"
echo "Second arg   \$2 = $2"
echo "Arg count    \$# = $#"
echo "All args     \$@ =" "$@"
i=1
for arg in "$@"; do            # "green tea" stays ONE word
    echo "  argument $i: [$arg]"
    i=$((i + 1))
done
```

```
student@okan-lab:~/lab/scripts$ ./args.sh apple "green tea" 42
Script name  $0 = ./args.sh
First arg    $1 = apple
Second arg   $2 = green tea
Arg count    $# = 3
All args     $@ = apple green tea 42
  argument 1: [apple]
  argument 2: [green tea]
  argument 3: [42]
student@okan-lab:~/lab/scripts$ ./args.sh
```

```
Script name  $0 = ./args.sh
First arg    $1 =
Second arg   $2 =
Arg count    $# = 0
All args     $@ =
student@okan-lab:~/lab/scripts$ ./greet.sh
What is your name? Ayse
Which year were you born? 2006
Nice to meet you, Ayse. You are about 20 years old.
```

Değişken	Anlamı
`$0`	betiğin, yazıldığı biçimdeki adı
`$1` … `$9`	1. … 9. argüman (10. için `${10}`)
`$#`	argüman sayısı
`"$@"`	tüm argümanlar, her biri ayrı bir sözcük olarak (döngülerde kullanın)
`$*`	tüm argümanlar, tek bir dizgide birleştirilmiş
`$?`	son komutun çıkış durumu (Bölüm 5)
`$$`	geçerli kabuğun PID'si (13. hafta)

Kabuk, betik başlamadan *önce* komut satırını sözcüklere ayırır; bu yüzden `"green tea"` tek bir argüman olarak gelir: `$#` 4 değil, 3'tür. Argüman yoksa `$1` boştur ve `$#` 0'dır; iyi yazılmış betiklerin önce `$#` değerini denetlemesinin nedeni budur. Yukarıdaki döküm bu alt bölümün son gösterimini, `read` komutunu da içerir.

4.5 `read` ile klavyeden girdi

```
#!/bin/bash
# greet.sh -- reading input from the keyboard with read
read -r -p "What is your name? " name
read -r -p "Which year were you born? " year
age=$(( $(date +%Y) - year ))
echo "Nice to meet you, $name. You are about $age years old."
```

Dökümde kullanıcı `Ayse` ve `2006` yazdı; yaş $2026 - 2006 = 20$'dir. `read -r var` bir girdi satırı bekler ve onu saklar; `-p` önce bir istem yazdırır; `-r` ters eğik çizgileri yazıldığı gibi tutar (her zaman kullanın). Birden çok adla `read -r first last` satırı boşluklardan böler.

Argümanlar mı, `read` mi? Otomasyon için argümanlar daha iyidir: `./grade.sh 72` başka bir betikten ya da klavye başında kimse yokken `cron`'dan çağrılabilir. `read` menüler ve oyunlar gibi etkileşimli araçlar içindir.

5 Çıkış Durumu ve Testler

5.1 Her komut evet ya da hayır diye yanıt verir

Bir süreç bittiğinde ebeveynine küçük bir sayı, **çıkış durumunu** (exit status) döndürür (C'de `main`'den `return n` ya da `exit(n)`, 3. hafta; ebeveyn bunu `wait()` ile toplar). Kabuk bu sayıyı `$?` içinde saklar:

```
student@okan-lab:~/lab/scripts$ ls ../docs > /dev/null
student@okan-lab:~/lab/scripts$ echo $?
0
student@okan-lab:~/lab/scripts$ ls ../nothing
```

```
ls: cannot access '../nothing': No such file or directory
student@okan-lab:~/lab/scripts$ echo $?
2
student@okan-lab:~/lab/scripts$ nosuchcommand
bash: nosuchcommand: command not found
student@okan-lab:~/lab/scripts$ echo $?
127
student@okan-lab:~/lab/scripts$ grep -q ERROR ../logs/system.log
student@okan-lab:~/lab/scripts$ echo $?
0
student@okan-lab:~/lab/scripts$ grep -q PANIC ../logs/system.log
student@okan-lab:~/lab/scripts$ echo $?
1
student@okan-lab:~/lab/scripts$ [ 5 -gt 3 ]
student@okan-lab:~/lab/scripts$ echo $?
0
student@okan-lab:~/lab/scripts$ [ "abc" = "abd" ]
student@okan-lab:~/lab/scripts$ echo $?
1
```

`$?`	**Geleneksel anlamı**
0	başarı, “doğru”
1	genel hata, “yanlış” (ör. `grep`: bulunamadı)
2	yanlış kullanım ya da ciddi hata (ör. olmayan bir dosya için `ls`)
126	komut bulundu ama çalıştırılamıyor (Permission denied)
127	komut bulunamadı

0 başarı demektir

Kabukta 0 *başarıdır* (“doğru”), 1 ile 255 arasındaki her değer ise başarısızlıktır. Bu, 0’ın yanlış olduğu C’nin tam tersidir. Mantıklıdır da: başarmanın tek bir yolu, başarısız olmanın ise birçok yolu vardır ve sayı hangisi olduğunu söyleyebilir.

`$?` her zaman *son* komutu gösterir; bu yüzden ona hemen bakın — `echo`’nun kendisi onu yeniden 0 yapar. `grep -q` hiçbir şey yazdırmaz; yalnızca çıkış durumuyla yanıt verir ve bu onu kararlar için ideal kılar.

5.2 `test` ve `[ ]`

`test` sayıları, dizgileri ya da dosyaları karşılaştırır ve çıkış durumuyla yanıt verir. `[`, son argümanı `]` olması gereken `test`’in başka bir adıdır. Döküm, `[` işaretinin kabuğa gömülü (builtin) bir *komut* olduğunu gösterir. Bu tek olgu tüm boşluk kurallarını açıklar: `[ "$n" -gt 3 ]` içindeki `[`, `"$n"`, `-gt`, `3`, `]` beş sözcüğü `[` komutunun ayrı argümanlarıdır; bu yüzden boşluklarla ayrılmaları gerekir. `[$n` yazmak, bir komut olmayan `[5` sözcüğünü üretir.

Tür	**İşleç**	**Ne zaman doğru … (örnek)**
Sayılar	`-eq -ne`	eşit / eşit değil: `[ "$n" -eq 0 ]`
	`-lt -le`	küçük / küçük ya da eşit: `[ "$x" -lt 10 ]`
	`-gt -ge`	büyük / büyük ya da eşit: `[ "$s" -ge 50 ]`
Dizgiler	`= !=`	aynı metin / farklı metin: `[ "$a" = yes ]`

Tür	İşleç	Ne zaman doğru … (örnek)
	`-z -n`	dizgi boş / boş değil: `[ -z "$1" ]`
Dosyalar	`-e`	yol var (herhangi bir tür): `[ -e "$p" ]`
	`-f -d`	sıradan dosya / dizin: `[ -d ~/lab ]`
	`-r -w -x`	sizin için okunabilir / yazılabilir / çalıştırılabilir
	`-s`	dosya var ve boş değil
Mantık	`!`	değilleme: `[ ! -d "$dir" ]`
	`&& ||`	iki test arasında ve / veya: `[ -f a ] && [ -f b ]`

İki tuzak: sayılar *harflerle* (`-lt`, `-gt`), dizgiler simgelerle (=, `!=`) karşılaştırılır. `[ ]` içinde < ve > karşılaştırma değil, yönlendirmedir (12. hafta). Ayrıca `-eq` değerleri, = ise metni karşılaştırır: `[ 05 -eq 5 ]` doğru, `[ 05 = 5 ]` yanlıştır. (Bash daha hoşgörülü bir anahtar sözcük olan `[[ … ]]` yapısını da sunar; biz taşınabilir `[ ]` kullanıyoruz.)

6 Karar Verme: `if`

6.1 Bir `if` deyiminin yapısı

```
if TEST; then
    commands            # TEST başarılı olunca çalışır (çıkış durumu 0)
elif OTHER_TEST; then
    commands            # isteğe bağlı, istediğiniz kadar
else
    commands            # isteğe bağlı: yukarıdakilerin hiçbiri başarılı olmazsa
fi                      # tersten "if" bloğu bitirir
```

TEST *herhangi bir* komut olabilir: `[ … ]`, `grep -q`, `mkdir`, …. `then`'den önceki noktalı virgül gereklidir, çünkü `then` yeni bir komut başlatmalıdır; `then`'i bir sonraki satıra yazmak da olur. Bash testleri yukarıdan aşağıya dener ve yalnızca testi başarılı olan ilk dalı çalıştırır.

6.2 Örnek: `check.sh`

```
#!/bin/bash
# check.sh -- file tests ([ -f ], [ -d ], [ -e ]) and exit statuses
if [ $# -ne 1 ]; then                  # exactly one argument?
    echo "Usage: $0 PATH" >&2          # >&2 : message goes to stderr
    exit 2                             # 2 = wrong usage
fi
path=$1
if [ -f "$path" ]; then
    echo "$path: regular file, $(wc -l < "$path") lines"
elif [ -d "$path" ]; then
    echo "$path: directory"
elif [ -e "$path" ]; then
    echo "$path: exists, but is neither a file nor a directory"
else
    echo "$path: does not exist" >&2
    exit 1                             # 1 = failure
```

```
fi
exit 0                                  # 0 = success
```

```
student@okan-lab:~/lab/scripts$ ./check.sh ../docs/syllabus.txt
../docs/syllabus.txt: regular file, 15 lines
student@okan-lab:~/lab/scripts$ ./check.sh ../logs
../logs: directory
student@okan-lab:~/lab/scripts$ ./check.sh /dev/null
/dev/null: exists, but is neither a file nor a directory
student@okan-lab:~/lab/scripts$ ./check.sh nothing.txt
nothing.txt: does not exist
student@okan-lab:~/lab/scripts$ echo $?
1
student@okan-lab:~/lab/scripts$ ./check.sh
Usage: ./check.sh PATH
student@okan-lab:~/lab/scripts$ echo $?
2
```

Her tür yol farklı bir dala gider. `/dev/null` vardır (`-e`) ama bir *aygıt dosyasıdır* (9. hafta: `/dev`); bu yüzden ne `-f` ne de `-d` testini geçer. Betik üç iyi alışkanlığı da gösterir: `$#` değerini denetler ve bir *Usage* (kullanım) satırı yazdırır; hata iletilerini `>&2` ile standart hataya (stderr) gönderir (12. hafta), böylece `./check.sh x > out.txt` hatayı yine ekranda gösterir; ve çağıran bir betiğin test edebileceği ayrı çıkış kodları kullanır (0 başarı, 1 başarısızlık, 2 yanlış kullanım).

6.3 Sayısal testler: `grade.sh`

```
#!/bin/bash
# grade.sh -- numeric tests with if / elif / else
# Usage: ./grade.sh SCORE        (an example scale, not an official one)
score=$1
if [ -z "$score" ]; then               # -z : the string is empty
    echo "Usage: $0 SCORE (0-100)" >&2
    exit 2
fi
if [ "$score" -lt 0 ] || [ "$score" -gt 100 ]; then
    echo "Error: score must be between 0 and 100" >&2
    exit 1
elif [ "$score" -ge 85 ]; then
    letter=A
elif [ "$score" -ge 70 ]; then
    letter=B
elif [ "$score" -ge 50 ]; then
    letter=C
else
    letter=F
fi
echo "Score $score -> grade $letter"
```

```
student@okan-lab:~/lab/scripts$ ./grade.sh 92
Score 92 -> grade A
student@okan-lab:~/lab/scripts$ ./grade.sh 72
Score 72 -> grade B
student@okan-lab:~/lab/scripts$ ./grade.sh 49
```

```
Score 49 -> grade F
student@okan-lab:~/lab/scripts$ ./grade.sh 150
Error: score must be between 0 and 100
student@okan-lab:~/lab/scripts$ ./grade.sh
Usage: ./grade.sh SCORE (0-100)
```

`elif` testleri bir merdiven oluşturur ve sıraları önemlidir: `-ge 50` önce gelseydi 92 puan bir `C` alırdı. `||` "veya" demektir: ikinci test yalnızca ilki başarısız olursa çalışır. (Ölçek yalnızca bir örnektir, resmî bir not ölçeği değildir.)

6.4 Hata ayıklama

Üç araç hataların (bug) çoğunu bulur:

- `bash -n script.sh` hiçbir şey çalıştırmadan sözdizimini denetler (Bölüm 10).
- `bash -x script.sh` her komutu, genişletmeden *sonraki* hâliyle ve çalıştırmadan önce, `+` işaretiyle yazdırır. Betik içinde `set -x` … `set +x` yalnızca bir kısmı izler.
- Hata iletisini okumak: ileti betiğin adını, *satır numarasını* ve bash'in işleyemediği sözcüğü söyler.

```
student@okan-lab:~/lab/scripts$ ./grade.sh abc
./grade.sh: line 9: [: abc: integer expression expected
./grade.sh: line 9: [: abc: integer expression expected
./grade.sh: line 12: [: abc: integer expression expected
./grade.sh: line 14: [: abc: integer expression expected
./grade.sh: line 16: [: abc: integer expression expected
Score abc -> grade F
student@okan-lab:~/lab/scripts$ bash -x grade.sh 72
+ score=72
+ '[' -z 72 ']'
+ '[' 72 -lt 0 ']'
+ '[' 72 -gt 100 ']'
+ '[' 72 -ge 85 ']'
+ '[' 72 -ge 70 ']'
+ letter=B
+ echo 'Score 72 -> grade B'
Score 72 -> grade B
```

İzleme, `$score` değerinin 72 olduğunu, `-ge 85` testinin başarısız, `-ge 70` testinin başarılı olduğunu gösterir. `abc` ile yapılan çalıştırma gerçek bir hatayı ortaya çıkarır: her sayısal test `integer expression expected` diye yakınır ve yanlış sayılır; bu yüzden betik `else` dalına düşer ve `F` notu yazdırır. (9. satır iki kez görünür, çünkü `||` ile bağlanan iki test de denenir.) Çözüm, girdiyi kullanmadan önce doğrulamaktır, örneğin

```
if ! echo "$score" | grep -qx '[0-9]\{1,3\}'; then
    echo "Error: not a number: $score" >&2; exit 1
fi
```

Burada `grep -x`, satırın tamamının bir ile üç arasında rakamla eşleşmesini ister (12. haftanın düzenli ifadeleri).

7 Döngüler

7.1 `for`: her öğe için bir kez

```
#!/bin/bash
# loops.sh -- three kinds of for loop
for city in Istanbul Ankara Izmir; do       # 1) over a list of words
    echo "City: $city"
done

for i in {1..5}; do                          # 2) over a range of numbers
    echo -n "$i squared is $((i * i)); "
done
echo                                         # finish the line

for f in ~/lab/docs/*.txt; do                # 3) over files (wildcard)
    echo "$(basename "$f"): $(wc -l < "$f") lines"
done
```

```
student@okan-lab:~/lab/scripts$ ./loops.sh
City: Istanbul
City: Ankara
City: Izmir
1 squared is 1; 2 squared is 4; 3 squared is 9; 4 squared is 16; 5 squared is 25;
final report.txt: 2 lines
linux_quotes.txt: 5 lines
notes_01.txt: 1 lines
notes_02.txt: 1 lines
notes_03.txt: 1 lines
syllabus.txt: 15 lines
```

Genel biçim `for VAR in LIST; do …; done` şeklindedir. Liste şunlar olabilir:

- tek tek yazılmış sözcükler (`Istanbul Ankara Izmir`) ya da argümanlar `"$@"`;
- bir küme parantezi genişletmesi (brace expansion) `{1..5}` → `1 2 3 4 5` (10. hafta). Küme parantezleri değişken içeremez: `{1..$n}` çalışmaz; bunun yerine `$(seq 1 "$n")` ya da bir `while` döngüsü kullanın;
- `~/lab/docs/*.txt` gibi bir joker karakter (wildcard): kabuk onu eşleşen dosya adlarıyla değiştirir. Döngü değişkeni tırnaklı olduğu için (`"$f"`) `final report.txt` adı tek parça kalır;
- bir komutun çıktısı, `$(komut)` (boşluk ve satır sonlarından bölünür).

Çıktıda `1 lines` de görülüyor: betik tam olarak kendisine söyleneni yazar; “1 line” yazdırmak bir `if` gerektirirdi. Bash’te C tarzı bir döngü de vardır, `for ((i = 1; i <= 5; i++))`; 16. lab alıştırmasında kullanılır. Joker karakter doğrudan istemde de çalışır:

```
student@okan-lab:~/lab/scripts$ for f in ../photos/*.png ../photos/*.JPG; do echo "found $f";
    done
found ../photos/img013.png
found ../photos/IMG_014.JPG
```

7.2 `while`: bir test başarılı olduğu sürece

```
#!/bin/bash
# while.sh -- a counting loop, and a file read line by line
n=3
while [ "$n" -gt 0 ]; do                    # repeat WHILE the test is true
    echo "Countdown: $n"
    n=$((n - 1))
done
echo "Go!"

sum=0
count=0
while read -r x; do                         # one line of the file per turn
    sum=$((sum + x))
    count=$((count + 1))
done < ~/lab/data/numbers.txt               # the whole loop reads this file
echo "$count numbers, sum = $sum, average = $((sum / count)) (integer)"
```

```
student@okan-lab:~/lab/scripts$ ./while.sh
Countdown: 3
Countdown: 2
Countdown: 1
Go!
20 numbers, sum = 11325, average = 566 (integer)
```

İki kalıp kullanımların çoğunu karşılar. *Sayaçlı* döngü değişkenini her turda değiştirir; `n =$((n - 1))` satırını unutursanız döngü hiç bitmez (Ctrl+C, yani SIGINT ile durdurun, 13. hafta). *Okuma döngüsü*, `while read -r x; do …; done < file`, her turda bir satır okur, çünkü `done`'dan sonraki `< file` yönlendirmesi döngünün tamamını besler; dosyanın sonunda `read` başarısız olur ve döngü durur. Sayılar tutuyor: 20 değerin toplamı $11\,325$'tir ve $11\,325/20 = 566{,}25$; tamsayı aritmetiği bunu 566'ya keser. (`until TEST; do …; done`, test *başarısız* olduğu sürece döner.)

7.3 Fonksiyonlar

```
#!/bin/bash
# functions.sh -- define a block of commands once, call it many times
line() {                                    # no arguments
    echo "------------------------------------"
}
count_word() {                              # $1 = word, $2 = file
    local n                                 # local: exists only in here
    n=$(grep -c -- "$1" "$2")
    echo "$1 appears on $n lines of $(basename "$2")"
}
is_even() {                                 # the exit status is the answer
    [ $(( $1 % 2 )) -eq 0 ]
}
line
count_word ERROR ~/lab/logs/system.log
count_word WARN  ~/lab/logs/system.log
line
for k in 7 12; do
    if is_even "$k"; then echo "$k is even"; else echo "$k is odd"; fi
```

```
done
```

```
student@okan-lab:~/lab/scripts$ ./functions.sh
----------------------------------
ERROR appears on 13 lines of system.log
WARN appears on 28 lines of system.log
----------------------------------
7 is odd
12 is even
```

Fonksiyon (function), kullanılmadan önce `name() { …; }` ile tanımlanan ve bir komut gibi çağrılan, adı olan bir komut bloğudur. İçinde `$1`, `$2`, … betiğin değil, *fonksiyonun* argümanlarıdır. `local` bir değişkeni fonksiyona özel tutar. Bir fonksiyonun çıkış durumu son komutununkidir (ya da `return n` ile ayarlanır, 0–255); bu yüzden `is_even` doğrudan bir `if` içinde kullanılabilir. *Metin* döndürmek için onu `echo` ile yazdırın ve `$( )` ile yakalayın. Neden `local n` ayrı bir satırda? `local n =$(grep …)` yazmak, `grep`'in çıkış durumunu `local`'inkinin arkasına gizlerdi.

8 Sözdizimi Başvuru Tablosu

Yapı	Sözdizimi	Unutmayın
shebang	`#!/bin/bash`	yalnızca 1. satır
değişken	`name=value`, `"$name"`, `"${name}s"`	= çevresinde boşluk yok
komut yerine koyma	`today=$(date +%F)`	komutun çıktısı
aritmetik	`n=$((n + 1))`, `$((a % b))`	yalnızca tamsayı
argümanlar	`$0 $1 ... $9 $# "$@"`	$# denetleyin
girdi	`read -r -p "Name? " name`	her zaman `-r`
if	`if [ "$n" -gt 0 ]; then ...; elif ...; then ...; else ...; fi`	`[ ]` içinde boşluk
for (liste)	`for c in a b c; do echo "$c"; done`	
for (aralık)	`for i in {1..5}; do ...; done`	`{}` içinde değişken yok
for (dosyalar)	`for f in *.txt; do wc -l "$f"; done`	`"$f"` tırnaklayın
for (C tarzı)	`for ((i = 0; i < 5; i++)); do ...; done`	yalnızca bash
while	`while [ "$n" -gt 0 ]; do n=$((n - 1)); done`	değişkeni değiştirin
dosya okuma	`while read -r line; do ...; done < file`	< done'dan sonra
fonksiyon	`name() { local x=$1; echo "$x"; }`	önce tanımlayın
ve / veya	`mkdir -p d && echo ok`; `[ -f f ] || exit 1`	kısa `if`
exit	`exit 0` (başarı), `exit 1` (başarısızlık)	0–255
hata ayıklama	`bash -n s.sh` (sözdizimi), `bash -x s.sh` (izleme)	

İki kullanışlı genişletme (isteğe bağlı)

`${var:-default}`, `var` değişkeninin değerini, boş ya da tanımsızsa `default` değerini verir:

`n =${1:-7}` "ilk argüman ya da 7" demektir. 7., 8. ve 14. lab alıştırmaları bunu kullanır. `${#var}` , `var` içindeki dizginin uzunluğunu verir.

9 İki Gerçek Otomasyon Betiği

9.1 `backup.sh`: zaman damgalı bir arşiv

```
#!/bin/bash
# backup.sh -- timestamped .tar.gz backup of ~/lab/docs into ~/lab/backup
# Usage: ./backup.sh            (works from any folder)
lab="$HOME/lab"
src="docs"                                  # folder to save (inside $lab)
dest="$lab/backup"
stamp=$(date +%Y%m%d_%H%M%S)                # e.g. 20261012_143005
archive="$dest/docs_$stamp.tar.gz"

if [ ! -d "$lab/$src" ]; then
    echo "Error: $lab/$src not found" >&2
    exit 1
fi
mkdir -p "$dest"                            # no error if it exists
if tar -czf "$archive" -C "$lab" "$src"; then
    n=$(tar -tzf "$archive" | grep -vc '/$')    # count files, not folders
    echo "Backup created: $archive"
    echo "Files: $n, size: $(du -h "$archive" | cut -f1)"
else
    echo "Error: tar failed" >&2
    exit 1
fi
```

Satır satır:

- `~/lab` yerine `$HOME/lab`: tilde çift tırnak içinde genişletil*mez*, `$HOME` ise genişletilir.
- `date +%Y%m%d_%H%M%S` `20261012_143005` gibi bir ad üretir: (saniye düzeyinde) benzersizdir ve yıl önce geldiği için sıralanabilir.
- `[ ! -d ... ]` kaynak yoksa açık bir iletiyle erkenden durur.
- `mkdir -p` hedefi gerekirse oluşturur, varsa sessiz kalır.
- `tar -czf archive -C dir docs` g**z**ip ile sıkıştırılmış bir arşiv dosyası (**f**) oluşturur (**c**); `-C` önce `~/lab` dizinine geçer, böylece arşivin içindeki yollar `docs/` ile başlar. `if tar …` tar'ın kendi çıkış durumunu test eder.
- `tar -tzf` arşivi listeler; `grep -vc '/$'` / ile *bitmeyen* satırları, yani klasörleri değil dosyaları sayar; `du -h | cut -f1` yalnızca boyutu tutar.

```
student@okan-lab:~/lab/scripts$ ./backup.sh
Backup created: /home/student/lab/backup/docs_20261012_085013.tar.gz
Files: 7, size: 4.0K
student@okan-lab:~/lab/scripts$ ls -l ../backup
total 4
```

```
-rw-rw-r-- 1 student student 862 Oct 12 08:50 docs_20261012_085013.tar.gz
student@okan-lab:~/lab/scripts$ tar -tzf ../backup/docs_*.tar.gz
docs/
docs/notes_02.txt
docs/syllabus.txt
docs/.settings
docs/notes_01.txt
docs/notes_03.txt
docs/linux_quotes.txt
docs/final report.txt
student@okan-lab:~/lab/scripts$ sleep 2
student@okan-lab:~/lab/scripts$ ./backup.sh
Backup created: /home/student/lab/backup/docs_20261012_085015.tar.gz
Files: 7, size: 4.0K
student@okan-lab:~/lab/scripts$ ls ../backup
docs_20261012_085013.tar.gz  docs_20261012_085015.tar.gz
```

Her çalıştırma yeni bir arşiv oluşturur (ikinci çalıştırma iki saniye sonradır); böylece hiçbir şeyin üzerine yazılmaz. Arşivde 7 dosya vardır: altı görünür belge ve gizli `.settings` (`tar` gizli dosyaları da alır). `final report.txt` doğru arşivlenir, çünkü her değişken tırnaklıdır. Dosyalarınıza dokunmadan geçici bir klasöre geri yüklemek için: `tar -xzf ../backup/docs_<damga>.tar.gz -C /tmp`. `cron` ile, `crontab -e` içine yazılan `0 18 * * * $HOME/lab/scripts/backup.sh` satırı betiği her gün 18.00'de çalıştırırdı (sınav konusu değildir). 13. lab alıştırması eski yedeklerin silinmesini ekler.

9.2 `report.sh`: bir günlük dosyasını ve bir CSV dosyasını özetlemek

```
#!/bin/bash
# report.sh -- one-page summary of logs/access.log and data/students.csv
log="$HOME/lab/logs/access.log"
csv="$HOME/lab/data/students.csv"
for f in "$log" "$csv"; do
    if [ ! -r "$f" ]; then echo "Error: cannot read $f" >&2; exit 1; fi
done
echo "===== Lab report, $(date '+%Y-%m-%d %H:%M') ====="

echo "--- Web server: $(basename "$log") ---"
total=$(wc -l < "$log")
errors=$(awk '$9 >= 400' "$log" | wc -l)    # field 9 = status code
echo "Requests: $total, errors (4xx/5xx): $errors ($((100 * errors / total))%)"
echo "Top 3 pages:"
awk '{print $7}' "$log" | sort | uniq -c | sort -rn | head -3

echo "--- Students: $(basename "$csv") ---"
echo "Students: $(tail -n +2 "$csv" | wc -l)"
echo "Per department:"
tail -n +2 "$csv" | cut -d, -f3 | sort | uniq -c
awk -F, 'NR > 1 { m + = $4; f + = $5; n++ }
         END { printf "Average midterm %.1f, average final %.1f\n", m/n, f/n }' "$csv"
passed=$(awk -F, 'NR > 1 && 0.4*$4 + 0.6*$5 >= 50' "$csv" | wc -l)
echo "Passed (0.4 x midterm + 0.6 x final >= 50): $passed"
```

```
student@okan-lab:~/lab/scripts$ ./report.sh
===== Lab report, 2026-10-12 08:50 =====
--- Web server: access.log ---
```

```
Requests: 300, errors (4xx/5xx): 98 (32%)
Top 3 pages:
     41 /courses/mcmp117
     37 /about.html
     33 /index.html
--- Students: students.csv ---
Students: 40
Per department:
     10 CS
     17 ECT
      2 EE
     11 ME
Average midterm 67.0, average final 65.6
Passed (0.4 x midterm + 0.6 x final >= 50): 35
```

Betik 12. haftanın boru hatlarını yeniden kullanır: awk `'{print $7}'` bir alanı çıkarır, `sort | uniq -c | sort -rn` sayar ve sıralar, `tail -n +2` CSV başlığını atlar ve `cut -d, -f3` bölüm sütununu alır. Yeni olan, awk'ın *ondalıklı* aritmetik yapmasıdır: `m += $4` vize sütununu toplar, `END` bloğu sonda bir kez çalışır ve `printf "%.1f"` tek ondalık basamak yazar. `0.4*$4 + 0.6*$5 >= 50` gibi tek başına bir koşul bir awk *örüntüsüdür* (pattern): awk yalnızca bu koşulun doğru olduğu satırları yazdırır ve `wc -l` bunları sayar. ($40/60$ ağırlıklandırması yalnızca bir örnektir.)

Raporun sağlaması

Tüm sayılar aynı dosyalardan Python ile bağımsız olarak yeniden hesaplandı:

- 300 isteğin 98'inin durum kodu ≥ 400: $100 \cdot 98/300 = 32{,}67$; bash tamsayı aritmetiği 32 yazar.
- `/index.html` ve `/images/logo.png` 33'er istek aldı; ilk 3'e yalnızca biri sığar ve eşitliği `sort` bozar.
- Bölümler: $17 + 11 + 10 + 2 = 40$ öğrenci.
- Vize ortalaması $66{,}95$ (yuvarlanarak `67.0` yazılır), final ortalaması $65{,}6$.
- 40 öğrencinin 35'i $0{,}4\,m + 0{,}6\,f \geq 50$ koşulunu sağlar.

10 Sık Görülen Hatalar ve Anlamları

`code/demo_errors.sh` betiği bozuk beş betik oluşturur. Bunların gerçek hata iletileri:

```
student@okan-lab:~/lab/scripts$ cat bad_space.sh
#!/bin/bash
name = "Ayse"
echo "Hello $name"
student@okan-lab:~/lab/scripts$ ./bad_space.sh
./bad_space.sh: line 2: name: command not found
Hello
student@okan-lab:~/lab/scripts$ cat bad_test.sh
#!/bin/bash
n=5
if [$n -gt 3]; then echo "big"; fi
student@okan-lab:~/lab/scripts$ ./bad_test.sh
./bad_test.sh: line 3: [5: command not found
```

```
student@okan-lab:~/lab/scripts$ cat bad_quotes.sh
#!/bin/bash
answer=""
if [ $answer = yes ]; then echo "OK"; fi
student@okan-lab:~/lab/scripts$ ./bad_quotes.sh
./bad_quotes.sh: line 3: [: =: unary operator expected
student@okan-lab:~/lab/scripts$ cat bad_if.sh
#!/bin/bash
if [ -d ~/lab ]; then
    echo "lab exists"
student@okan-lab:~/lab/scripts$ bash -n bad_if.sh
bad_if.sh: line 4: syntax error: unexpected end of file
```

= çevresinde boşluk `name: command not found`. Bash `name` adlı bir komut çalıştırdı. Betiğin durma*dığına* dikkat edin: devam etti ve boş bir değişkenle `Hello` yazdırdı. Düzeltme: `name ="Ayse"`. Tersi bir hata olan `name = Ayse`, `name` değişkeni boşken `Ayse` komutunu çalıştırır.

`[ ]` içinde boşluk yok `[5: command not found`. Düzeltme: `if [ "$n" -gt 3 ]; then`.

Eksik tırnaklar `[: =: unary operator expected`. Boş ve tırnaksız `$answer` kayboldu ve geriye `[ = yes ]` kaldı. Tırnaklarla `[ "" = yes ]` yalnızca yanlıştır. Boşluklu dosya adları tırnaksız kullanılırsa her yarı için `No such file or directory` alınır (Bölüm 4.3).

Eksik `fi`, `done` ya da tırnak `syntax error: unexpected end of file`: bir blok açıldı ve hiç kapatılmadı. `bash -n` bunu betiği çalıştırmadan bulur. Bildirilen satır dosyanın *sonudur*; asıl hata daha öncedir.

Eksik `chmod +x` `Permission denied`, çıkış durumu 126 (Bölüm 3.2).

Sayı değil `-lt`, `-gt`, … işleçlerinden `integer expression expected` (Bölüm 6.4).

10.1 Windows satır sonları (CRLF)

Windows düzenleyicileri her satırı iki karakterle, satır başı ve satır besleme ile bitirir (`\r\n`, “CRLF”); Linux yalnızca `\n` kullanır. Not Defteri’nde yazılıp Linux’a kopyalanan bir betiğin her satırının sonunda bu yüzden görünmez bir `\r` bulunur ve bu karakter satırın son sözcüğüne yapışır:

```
student@okan-lab:~/lab/scripts$ ./win.sh
bash: ./win.sh: cannot execute: required file not found
student@okan-lab:~/lab/scripts$ bash win.sh
win.sh: line 2: cd: $'/home/student/lab\r': No such file or directory
Hello from Windows
student@okan-lab:~/lab/scripts$ file win.sh
win.sh: Bourne-Again shell script, ASCII text executable, with CRLF line terminators
student@okan-lab:~/lab/scripts$ cat -A win.sh
#!/bin/bash^M$
cd ~/lab^M$
echo "Hello from Windows"^M$
student@okan-lab:~/lab/scripts$ sed -i 's/\r$//' win.sh
student@okan-lab:~/lab/scripts$ ./win.sh
Hello from Windows
```

- `./win.sh`: çekirdek tam olarak `/bin/bash\r` adlı, var olmayan bir yorumlayıcı arar. Bash 5.2 (burada kullanılan) `cannot execute: required file not found` bildirir; bash’in eski

sürümleri (ör. Ubuntu 22.04) klasik `/bin/bash^M: bad interpreter: No such file or directory` iletisini verir. İkisi de aynı anlama gelir.

- `bash win.sh` çalışır, ama `cd` sonunda `\r` bulunan yolu alır (`$'…\r'` olarak gösterilir) ve başarısız olur.
- `file` `with CRLF line terminators` der; `cat -A` her `\r` karakterini satır sonu işareti `$`'dan önce `^M` olarak gösterir.
- Düzeltme: `sed -i 's/\r$//' win.sh` her satırın sonundaki `\r` karakterini siler; kuruluysa `dos2unix win.sh` aynı işi yapar. VS Code'da durum çubuğundaki "CRLF" yazısına tıklayıp "LF"yi seçin.

11 Özet

Sekiz cümlede 14. hafta

1. Kabuk betiği, 1. satırında `#!/bin/bash` bulunan, komutlardan oluşan bir metin dosyasıdır; `chmod +x` sonrası `./script.sh` ile, fork ve exec ile oluşturulan bir çocuk süreçte çalışır.
2. Değişkenler boşluksuz atanır (`name =value`) ve çift tırnak içinde kullanılır (`"$name"`); tek tırnaklar her şeyi olduğu gibi bırakır.
3. `$(komut)` bir komutun çıktısını yerleştirir, `$((ifade))` tamsayılarla hesap yapar.
4. Argümanlar `$1` … `$9` olarak, `$#` ve `"$@"` ile birlikte gelir; `read -r` klavyeyi okur.
5. Her komut bir çıkış durumu döndürür; 0 başarı demektir; `exit n` bir betiğin durumunu ayarlar.
6. `[ ]` sayıları (`-eq -lt -gt` …), dizgileri (`= != -z`) ve dosyaları (`-e -f -d`) test eden bir komuttur; `if / elif / else / fi` bir dal seçer.
7. `for` sözcükler, aralıklar ve dosyalar üzerinde döner; `while` bir test başarılı olduğu sürece tekrarlar ya da bir dosyayı satır satır okur; fonksiyonlar bir komut bloğuna ad verir.
8. `bash -n` ve `bash -x` hata ayıklamaya yardım eder; değişkenlerinizi tırnaklayın ve betikleri LF satır sonlarıyla kaydedin.

12 Sözlükçe

Argüman (konumsal parametre) Betik adından sonra verilen bir sözcük; betiğin içinde `$1`, `$2`, …

Aritmetik genişletme (arithmetic expansion) `$(( ))`: kabuğun yaptığı tamsayı hesabı.

Betik (shell script) Bash gibi bir yorumlayıcının çalıştırdığı, kabuk komutlarından oluşan metin dosyası.

Çıkış durumu (exit status) Her komutun döndürdüğü 0–255 arası sayı; 0 = başarı; `$?` içinde.

CRLF Windows satır sonu `\r\n`; Linux'ta betikleri bozar.

Fonksiyon (function) Kendi argümanları olan, adı verilmiş komut bloğu.

Komut yerine koyma (command substitution) `$( )`: içindeki komutun çıktısıyla değiştirilir.

PATH Kabuğun komutları aradığı dizinleri listeleyen değişken.

Shebang Çekirdeğe betiği hangi programın çalıştıracağını söyleyen ilk satır `#!yorumlayıcı`.

test / [ ] Bir koşulu değerlendirip çıkış durumuyla yanıt veren komut.

Tırnaklama (quoting) Çift tırnak boşlukları korur ve değişkenleri genişletir; tek tırnak her şeyi olduğu gibi bırakır.

13 Kendiniz Deneyin

Kendi makinenizde keşfedin

1. Bir betiğe `cd /tmp` ve `pwd` yazıp çalıştırın. Sonrasında terminaliniz hangi dizinde? Şimdi betiği `source` ile çalıştırın. Farkı 13. haftayla (süreçler) açıklayın.
2. `hello.sh` dosyasına `echo "My PID is $$, my parent is $PPID"` satırını ekleyin, iki kez çalıştırın ve istemdeki `echo $$` ile karşılaştırın.
3. `bash -x loops.sh` komutunu çalıştırın. Her döngünün gövdesi kaç kez çalışıyor?
4. `grade.sh`'nin sayısal olmayan girdiyi reddetmesini sağlayın (Bölüm 6.4'deki `grep -qx` testini kullanın).
5. `shellcheck` kuruluysa, iki `# shellcheck` satırını sildikten sonra onu `quotes.sh` üzerinde çalıştırın. Ne bildiriyor?

14 Tekrar Soruları

Kavramsal

1. `#!/bin/bash` satırı ne yapar ve onu kim okur?
2. `./hello.sh` neden çalıştırma izni gerektirirken `bash hello.sh` gerektirmez?
3. Bir betik `cd /tmp` içeriyor. Betik bittikten sonra terminaliniz hâlâ ev dizininizde. Neden?
4. `"$name"`, `'$name'` ve `$name` arasındaki farkı açıklayın.
5. Neden `[ ]` içinde boşluk olmalı, ama bir atamada `=` çevresinde boşluk olmamalıdır?
6. Başarılı bir komut hangi çıkış durumunu döndürür? 126 ve 127 ne anlama gelir?
7. `read` ne zaman doğru araçtır, argümanlar ne zaman daha iyidir?
8. Windows'tan kopyalanan bir betik için `bad interpreter` (ya da `required file not found`) genellikle ne anlama gelir ve nasıl düzeltilir?

Sayısal ve uygulamalı

9. `echo $(( (17 + 3) / 6 * 6 ))` ne yazdırır? Ya `echo $(( 17 % 5 ** 2 ))`?
10. `./args.sh "a b" c "" d`: `$#`, `$1` ve `$3` nedir?

11. Bir `while` döngüsü `n =10` ile başlıyor ve `[ "$n" -gt 0 ]` olduğu sürece `n =$((n - 3))` çalıştırıyor. Gövde kaç kez çalışır ve sonda n kaçtır?
12. `[ 010 -eq 10 ]`, `[ 010 = 10 ]` ve `[ -z "" ]` testleri hangi çıkış durumunu döndürür?
13. `s =0` iken `for i in {1..4}; do s =$((s + i * i)); done`: s kaçtır?
14. `~/lab/backup` klasörü varsa `OK`, yoksa `MISSING` yazdıran tek bir satır yazın.

Sayısal ve uygulamalı soruların yanıtları

9. $(17+3)/6 = 20/6 = 3$ (tamsayı), $3 \times 6 = 18$. İkincisinde ** (üs alma) %'den daha sıkı bağlanır: $5^2 = 25$ ve 17 mod $25 = 17$.
10. $\$\# = 4$ (boş dizgi `""` de bir argümandır), $\$1 =$ `a  b`, $\$3$ boştur.
11. n: $10 \to 7 \to 4 \to 1 \to -2$; gövde 4 kez çalışır ve sonda $n = -2$'dir.
12. 0 (sayısal olarak eşit), 1 (farklı metin), 0 (dizgi boş).
13. $s = 1 + 4 + 9 + 16 = 30$.
14. `[ -d ~/lab/backup ] && echo OK || echo MISSING` (ya da bir `if/else/fi`).

15 Sırada Ne Var: 15. Hafta

- **15. hafta: Final sınavı tekrarı.** Final sınavı dersin tamamını kapsar: 1–7. haftaların kuramı (8. haftanın çalışma kılavuzunu kullanın) ve uygulamalı Linux haftaları 9–14 — dizinlerde gezinme, dosya işlemleri, izinler, metin işleme ve boru hatları, süreçler ve iş denetimi, kabuk betikleri.
- "Komutu / boru hattını / betiği yazın" türünden sorular bekleyin: aşağıdaki Lab Föyü en iyi hazırlıktır.
- Sınav tarihi, saati ve salonu dersin Telegram kanalında duyurulacaktır.

İleri okuma (isteğe bağlı)

- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley: Bölüm 2.2 (kullanıcı ve işletim sistemi arayüzü: komut yorumlayıcıları, kabuklar) ve Bölüm 3.3 (`fork()` ve `exec()` ile süreç oluşturma).
- R. H. ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, 5. bölüm, "Interlude: Process API" (bir kabuğun fork, exec ve wait'i nasıl kullandığı): `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-api.pdf`
- GNU Bash başvuru kılavuzu: `https://www.gnu.org/software/bash/manual/bash.html`
- W. Shotts, *The Linux Command Line*, 4. kısım "Writing Shell Scripts" (ücretsiz PDF): `https://linuxcommand.org/tlcl.php`
- ShellCheck, kabuk betiklerindeki hataları bulur: `https://www.shellcheck.net`

16 Lab Föyü: Notlandırılan Betik Yazma Alıştırmaları

Her betiği `~/lab/scripts` içinde, temiz bir laboratuvar klasöründen başlayarak yazın; klasörü şu komut yeniden oluşturur: `bash ~/mcmp117_lab_setup.sh`. Her betik `#!/bin/bash` ile başlamalı, ne yaptığını söyleyen bir yorum satırı içermeli, çalıştırılabilir olmalı, `bash -n` denetiminden geçmeli ve değişkenlerini tırnaklamalıdır. Betikleri gösterildiği gibi `exNN_ad.sh` biçiminde adlandırın. Derste 2., 6., 11. ve 13. alıştırmalar yapılır ve notlandırılır ("Sınıf İçi Lab" slaytı); geri kalanlar ev ödevi ve sınav alıştırmasıdır. Toplam: 100 puan. Her alıştırma için önerilen değerlendirme ölçütü: hatasız çalışma $\%40$, doğru çıktı $\%30$, hatalı girdiyi ele alma $\%15$, biçem (yorumlar, tırnaklar, shebang) $\%15$.

A Kısmı --- Temeller (her biri 5 puan)

1. **`ex01_myinfo.sh`** Kullanıcı adınızı, ev dizininizi, bugünün tarihini `YYYY-MM-DD` biçiminde, oturum açma kabuğunuzu ve `~/lab/data/students.csv` dosyasındaki öğrenci sayısını (satır sayısı eksi başlık) yazdırın.
2. **`ex02_power.sh N`** N, N^2 ve N^3 değerlerini yazdırın. Tam olarak bir argüman verilmezse stderr'e bir Usage satırı yazdırın ve 2 durumuyla çıkın.
3. **`ex03_hello_name.sh NAME [CITY]`** `Hello NAME from CITY!` yazdırın; şehir verilmezse Istanbul kullanın; ad verilmezse bir Usage satırı yazdırın ve 1 ile çıkın.
4. **`ex04_calc.sh`** `read` ile iki tamsayı okuyun ve bunların toplamını, farkını, çarpımını ve kalanıyla birlikte tamsayı bölümünü yazdırın. Sıfıra bölmeyi reddedin.

B Kısmı --- Kararlar ve basit döngüler (her biri 6 puan)

5. **`ex05_sign.sh N`** N'nin pozitif, negatif ya da sıfır olduğunu ve tek mi çift mi olduğunu söyleyin.
6. **`ex06_what.sh PATH…`** Her argüman için *file* (satır sayısıyla), *directory* ya da *missing* yazdırın. En az bir yol yoksa 1, yoksa 0 durumuyla çıkın.
7. **`ex07_table.sh [N]`** Bir `for` döngüsüyle $N \times 1 \ldots N \times 10$ çarpım tablosunu yazdırın (varsayılan $N = 7$).
8. **`ex08_sum.sh [N]`** Bir `while` döngüsüyle $1 + 2 + \cdots + N$ toplamını hesaplayın (varsayılan $N = 100$) ve $\frac{N(N+1)}{2}$ formülüyle karşılaştırın.

C Kısmı --- Dosyalar ve veriler (her biri 7 puan)

9. **`ex09_doclines.sh`** `~/lab/docs` içindeki her `.txt` dosyası için adını ve satır sayısını yazdırın; en sonda toplamı ve en uzun dosyayı yazdırın.
10. **`ex10_minmax.sh`** `~/lab/data/numbers.txt` dosyasını bir `while read` döngüsüyle okuyun ve sayıyı, en küçüğü, en büyüğü, toplamı ve tamsayı ortalamayı — `sort` kullanmadan — yazdırın.
11. **`ex11_levels.sh`** `~/lab/logs/system.log` dosyasındaki `INFO`, `WARN` ve `ERROR` satırlarını bir `for` döngüsü ve `grep -c` ile sayın ve her sayıyı tüm satırların yüzdesi olarak yazdırın.
12. **`ex12_temp.sh`** Santigradı Fahrenhayta çeviren bir `c2f` fonksiyonu yazın, $F = \frac{9}{5}C + 32$, ve $-10, 0, 10, \ldots, 40$ °C için bir tablo yazdırın.

D Kısmı --- Zorlu alıştırmalar (her biri 7 puan)

13. **`ex13_backup_keep.sh`** `backup.sh`'yi, `~/lab/backup` içinde yalnızca en yeni 3 arşiv kalacak biçimde genişletin; sildiği her arşivin adını yazdırsın. Bir saniye arayla beş kez çalıştırarak deneyin.
14. **`ex14_guess.sh [SECRET]`** Bir tahmin oyunu: gizli sayı 1 ile 100 arasındadır (argüman ya da `$RANDOM` ile rastgele); doğru tahmine kadar tahminleri okuyun, *Too small* / *Too big* diye yanıt verin ve en sonda deneme sayısını yazdırın.
15. **`ex15_dept.sh DEPT`** `students.csv` dosyasından bir bölümün öğrenci sayısını ve final notu ortalamasını (tek ondalık) yazdırın; bilinmeyen bir bölüm için bir iletiyle 1 durumuyla çıkın.
16. **`ex16_status_bars.sh`** `access.log` içindeki her HTTP durum kodu için kodu, sayısını ve her 5 istek için bir `#` içeren (aşağı yuvarlanmış) bir çubuk yazdırın.

Puanlar: $4 \times 5 + 4 \times 6 + 8 \times 7 = 20 + 24 + 56 = 100$.

17 Lab Föyü Çözümleri

Bu çözümler `code/solutions/` klasöründedir. Her biri temiz bir laboratuvar klasöründe bash 5.2 ile çalıştırıldı ve `bash -n` ile ShellCheck denetimlerinden geçer. Çıktılar gerçektir; hepsi `code/solutions/out_solutions.txt` dosyasında toplanmıştır. Elbette başka doğru çözümler de olabilir. Önce deneyin, sonra karşılaştırın!

1. ex01_myinfo.sh

```
#!/bin/bash
# ex01_myinfo.sh -- Lab exercise 1: information about me and my lab
echo "User:      $(whoami)"
echo "Home:      $HOME"
echo "Date:      $(date +%Y-%m-%d)"
echo "Shell:     $SHELL"
students=$(( $(wc -l < ~/lab/data/students.csv) - 1 ))   # minus the header line
echo "Students in students.csv: $students"
```

```
student@okan-lab:~/lab/scripts$ ./ex01_myinfo.sh
User:      student
Home:      /home/student
Date:      2026-10-12
Shell:     /bin/bash
Students in students.csv: 40
```

2. ex02_power.sh

```
#!/bin/bash
# ex02_power.sh -- Lab exercise 2: square and cube of a number
if [ $# -ne 1 ]; then
    echo "Usage: $0 NUMBER" >&2
    exit 2
fi
n=$1
echo "n = $n, n^2 = $((n * n)), n^3 = $((n * n * n))"
```

```
student@okan-lab:~/lab/scripts$ ./ex02_power.sh 12
n = 12, n^2 = 144, n^3 = 1728
student@okan-lab:~/lab/scripts$ ./ex02_power.sh
Usage: ./ex02_power.sh NUMBER
```

3. ex03_hello_name.sh

```
#!/bin/bash
# ex03_hello_name.sh -- Lab exercise 3: greeting with a required and an optional argument
if [ $# -lt 1 ]; then
    echo "Usage: $0 NAME [CITY]" >&2
    exit 1
fi
city=$2
if [ -z "$city" ]; then
    city="Istanbul"                 # default when no city is given
fi
echo "Hello $1 from $city!"
```

```
student@okan-lab:~/lab/scripts$ ./ex03_hello_name.sh Elif
Hello Elif from Istanbul!
student@okan-lab:~/lab/scripts$ ./ex03_hello_name.sh Can Izmir
Hello Can from Izmir!
```

4. ex04_calc.sh

```
#!/bin/bash
# ex04_calc.sh -- Lab exercise 4: read two integers and calculate
read -r -p "First integer:  " a
read -r -p "Second integer: " b
echo "$a + $b = $((a + b))"
echo "$a - $b = $((a - b))"
echo "$a * $b = $((a * b))"
if [ "$b" -eq 0 ]; then
    echo "Cannot divide by zero"
else
    echo "$a / $b = $((a / b)) remainder $((a % b))"
fi
```

```
student@okan-lab:~/lab/scripts$ ./ex04_calc.sh
First integer:  17
Second integer: 5
17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3 remainder 2
```

5. ex05_sign.sh

```
#!/bin/bash
# ex05_sign.sh -- Lab exercise 5: positive / negative / zero, and even / odd
if [ $# -ne 1 ]; then
    echo "Usage: $0 INTEGER" >&2
    exit 2
fi
n=$1
if [ "$n" -gt 0 ]; then
    sign="positive"
elif [ "$n" -lt 0 ]; then
    sign="negative"
else
    sign="zero"
fi
if [ $((n % 2)) -eq 0 ]; then parity="even"; else parity="odd"; fi
echo "$n is $sign and $parity"
```

```
student@okan-lab:~/lab/scripts$ ./ex05_sign.sh -8
-8 is negative and even
student@okan-lab:~/lab/scripts$ ./ex05_sign.sh 0
0 is zero and even
```

```
student@okan-lab:~/lab/scripts$ ./ex05_sign.sh
Usage: ./ex05_sign.sh INTEGER
```

6. ex06_what.sh

```
#!/bin/bash
# ex06_what.sh -- Lab exercise 6: classify every argument; exit 1 if one is missing
status=0
for p in "$@"; do
    if [ -f "$p" ]; then
        echo "$p: file ($(wc -l < "$p") lines)"
    elif [ -d "$p" ]; then
        echo "$p: directory"
    else
        echo "$p: missing"
        status=1
    fi
done
exit $status
```

```
student@okan-lab:~/lab/scripts$ ./ex06_what.sh ../docs/syllabus.txt ../src nothing.txt
../docs/syllabus.txt: file (15 lines)
../src: directory
nothing.txt: missing
student@okan-lab:~/lab/scripts$ echo $?
1
```

`status` değişkeni, döngü diğer argümanlarla sürerken bir başarısızlığı hatırlar.

7. ex07_table.sh

```
#!/bin/bash
# ex07_table.sh -- Lab exercise 7: multiplication table of N
n=${1:-7}                          # default 7 when no argument is given
for i in {1..10}; do
    echo "$n x $i = $((n * i))"
done
```

```
student@okan-lab:~/lab/scripts$ ./ex07_table.sh 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

8. ex08_sum.sh

```
#!/bin/bash
# ex08_sum.sh -- Lab exercise 8: 1 + 2 + ... + N with a while loop
n=${1:-100}
i=1
sum=0
while [ "$i" -le "$n" ]; do
    sum=$((sum + i))
    i=$((i + 1))
done
echo "Loop:    1 + ... + $n = $sum"
echo "Formula: n(n+1)/2   = $((n * (n + 1) / 2))"
```

```
student@okan-lab:~/lab/scripts$ ./ex08_sum.sh 100
Loop:    1 + ... + 100 = 5050
Formula: n(n+1)/2   = 5050
```

Sağlama: $\frac{100 \cdot 101}{2} = 5050$.

9. ex09_doclines.sh

```
#!/bin/bash
# ex09_doclines.sh -- Lab exercise 9: lines per document, total, longest
total=0
max=0
longest=""
for f in ~/lab/docs/*.txt; do
    n=$(wc -l < "$f")
    printf '%-18s %3d\n' "$(basename "$f")" "$n"
    total=$((total + n))
    if [ "$n" -gt "$max" ]; then
        max=$n
        longest=$(basename "$f")
    fi
done
echo "Total: $total lines; longest: $longest ($max lines)"
```

```
student@okan-lab:~/lab/scripts$ ./ex09_doclines.sh
final report.txt     2
linux_quotes.txt     5
notes_01.txt         1
notes_02.txt         1
notes_03.txt         1
syllabus.txt        15
Total: 25 lines; longest: syllabus.txt (15 lines)
```

`printf '%-18s %3d'` adı 18 sütunda sola, sayıyı 3 sütunda sağa yaslı yazar. Toplam: $2+5+1+1+1+15 = 25$.

10. ex10_minmax.sh

```
#!/bin/bash
# ex10_minmax.sh -- Lab exercise 10: minimum, maximum, sum, average of numbers.txt
file=~/lab/data/numbers.txt
count=0
sum=0
while read -r x; do
    if [ "$count" -eq 0 ]; then min=$x; max=$x; fi   # first number
    if [ "$x" -lt "$min" ]; then min=$x; fi
    if [ "$x" -gt "$max" ]; then max=$x; fi
    sum=$((sum + x))
    count=$((count + 1))
done < "$file"
echo "count=$count min=$min max=$max sum=$sum average=$((sum / count))"
```

```
student@okan-lab:~/lab/scripts$ ./ex10_minmax.sh
count=20 min=280 max=998 sum=11325 average=566
```

İlk sayı hem `min` hem `max` için başlangıç değeridir. Toplam `while.sh` ile aynıdır: 11 325, ortalama 566 (tamsayı).

11. ex11_levels.sh

```
#!/bin/bash
# ex11_levels.sh -- Lab exercise 11: messages per level in system.log
log=~/lab/logs/system.log
total=$(wc -l < "$log")
for level in INFO WARN ERROR; do
    n=$(grep -c " $level " "$log")
    echo "$level: $n of $total lines ($((100 * n / total))%)"
done
```

```
student@okan-lab:~/lab/scripts$ ./ex11_levels.sh
INFO: 79 of 120 lines (65%)
WARN: 28 of 120 lines (23%)
ERROR: 13 of 120 lines (10%)
```

$79 + 28 + 13 = 120$. Yüzdelerin toplamı %100 değil %98'dir, çünkü tamsayı bölmesi keser ($65{,}8 \to 65$, $23{,}3 \to 23$, $10{,}8 \to 10$). `" $level "` içindeki boşluklar yalnızca düzey sütununun eşleşmesini sağlar.

12. ex12_temp.sh

```
#!/bin/bash
# ex12_temp.sh -- Lab exercise 12: a function that converts Celsius to Fahrenheit
c2f() {
    echo $(( $1 * 9 / 5 + 32 ))        # exact for multiples of 5
}
echo "Celsius  Fahrenheit"
for c in -10 0 10 20 30 40; do
    printf '%7d  %10d\n' "$c" "$(c2f "$c")"
done
```

```
student@okan-lab:~/lab/scripts$ ./ex12_temp.sh
Celsius  Fahrenheit
    -10          14
      0          32
     10          50
     20          68
     30          86
     40         104
```

5'in katları için $9C/5$ bir tamsayıdır; bu yüzden tamsayı sonucu kesindir: $40 \cdot 9/5 + 32 = 104$.

13. ex13_backup_keep.sh

```
#!/bin/bash
# ex13_backup_keep.sh -- Lab exercise 13: backup.sh that keeps only the 3 newest archives
lab="$HOME/lab"
dest="$lab/backup"
keep=3
archive="$dest/docs_$(date +%Y%m%d_%H%M%S).tar.gz"
mkdir -p "$dest"
tar -czf "$archive" -C "$lab" docs || exit 1
echo "Created $(basename "$archive")"
# names contain only letters, digits and _ , so ls is safe here; newest first
# shellcheck disable=SC2012
ls -1t "$dest"/docs_*.tar.gz | tail -n +$((keep + 1)) | while read -r old; do
    echo "Removing old backup $(basename "$old")"
    rm -- "$old"
done
```

```
student@okan-lab:~/lab/scripts$ for _ in 1 2 3 4 5; do ./ex13_backup_keep.sh; sleep 1; done
Created docs_20261012_085022.tar.gz
Created docs_20261012_085023.tar.gz
Created docs_20261012_085024.tar.gz
Created docs_20261012_085025.tar.gz
```

```
Removing old backup docs_20261012_085022.tar.gz
Created docs_20261012_085026.tar.gz
Removing old backup docs_20261012_085023.tar.gz
student@okan-lab:~/lab/scripts$ ls ../backup
docs_20261012_085024.tar.gz  docs_20261012_085025.tar.gz  docs_20261012_085026.tar.gz
```

`ls -1t` arşivleri en yeniden başlayarak listeler; `tail -n +4` en yeni üçünü atlar ve gerisini döngüye verir. **Dikkat:** `rm` komutunun geri dönüşüm kutusu yoktur (10. hafta). Böyle bir betiği önce kopyalar üzerinde deneyin ve sildiğini yazdırın. `ls` çıktısını ayrıştırmak burada yalnızca adlar boşluk içermediği için güvenlidir.

14. ex14_guess.sh

```
#!/bin/bash
# ex14_guess.sh -- Lab exercise 14: guess the secret number (1-100)
secret=${1:-$((RANDOM % 100 + 1))}       # fixed secret if given (for testing)
tries=0
guess=0
while [ "$guess" -ne "$secret" ]; do
    read -r -p "Your guess: " guess
    tries=$((tries + 1))
    if [ "$guess" -lt "$secret" ]; then
        echo "Too small"
    elif [ "$guess" -gt "$secret" ]; then
        echo "Too big"
    fi
done
echo "Correct! You needed $tries tries."
```

```
student@okan-lab:~/lab/scripts$ ./ex14_guess.sh 37
Your guess: 50
Too big
Your guess: 25
Too small
Your guess: 37
Correct! You needed 3 tries.
```

Oyuncu 50, 25 ve 37 yazdı. Sabit bir gizli sayıyla oyun test edilebilir; argüman verilmezse `$RANDOM % 100 + 1` 1–100 arasında bir değer verir.

15. ex15_dept.sh

```
#!/bin/bash
# ex15_dept.sh -- Lab exercise 15: students and average final of one department
csv=~/lab/data/students.csv
dept=$1
if [ -z "$dept" ]; then
    echo "Usage: $0 DEPARTMENT" >&2
    exit 2
fi
n=$(cut -d, -f3 "$csv" | grep -cx "$dept")
if [ "$n" -eq 0 ]; then
    echo "No students in department $dept" >&2
    exit 1
fi
avg=$(awk -F, -v d="$dept" '$3 == d { s += $5; k++ } END { printf "%.1f", s/k }' "$csv")
echo "$dept: $n students, average final $avg"
```

```
student@okan-lab:~/lab/scripts$ ./ex15_dept.sh ECT
ECT: 17 students, average final 63.1
student@okan-lab:~/lab/scripts$ ./ex15_dept.sh EE
EE: 2 students, average final 63.0
student@okan-lab:~/lab/scripts$ ./ex15_dept.sh XYZ
No students in department XYZ
```

`grep -cx` bölüm adının *tamamıyla* eşleşen satırları sayar; `awk -v d =…` kabuk değişkenini awk'a aktarır. Sağlama: ECT final notlarının toplamı 1072, $1072/17 = 63{,}06$.

16. ex16_status_bars.sh

```
#!/bin/bash
# ex16_status_bars.sh -- Lab exercise 16: bar chart of HTTP status codes (# = 5 requests)
log=~/lab/logs/access.log
awk '{print $9}' "$log" | sort | uniq -c | while read -r count code; do
    bar=""
    for ((i = 0; i < count / 5; i++)); do bar="$bar#"; done
    printf '%s %4d %s\n' "$code" "$count" "$bar"
done
```

```
student@okan-lab:~/lab/scripts$ ./ex16_status_bars.sh
200  148 ############################
301   20 ####
304   34 ######
403   17 ###
404   57 ###########
500   24 ####
```

`while read -r count code`, `uniq -c` çıktısının her satırını iki değişkene böler. Sağlama: $148/5 = 29{,}6 \to 29$ karakter; sayıların toplamı 300'dür.